

coverage_matrix

Test-Coverage Assessment — digital.vasic.translator

Date: 2026-06-12 **Module:** digital.vasic.translator (Go 1.26.2, darwin/arm64) **Scope:** main module only (cmd/ pkg/ internal/ api/ test/ tests/). Git submodules (challenges/, containers/, helix_qa/, llms_verifier/, doc_processor/, llm_provider/, vision_engine/, security/, etc.) are SEPARATE Go modules and are NOT counted here.

Headline numbers (all captured from real runs)

- **Total statement coverage: 50.7%**
- **Suite status: FAIL** — one package (pkg/security) has a failing test.
- Packages reporting **0.0% / no test files: 27** (within cmd/ pkg/ internal/ api/ tools/).
- Packages that **FAIL/error: 1** (pkg/security, test failure — compiles fine).

Commands used (verbatim)

```
go test -cover ./... 2>&1 | tee /tmp/_cov_run.txt
go test -coverprofile=/tmp/cov.out ./... 2>&1 | tail
go tool cover -func=/tmp/cov.out | tail -1
```

Captured go tool cover -func total line (evidence)

total:
(statements) 50.7%

NOTE: 50.7% is the aggregate across all packages that produced a coverage profile, including the many 0.0% / no-test packages — it is the honest whole-module figure, not a cherry-picked subset.

Per-package coverage matrix (worst-first)

Source: /tmp/_cov_run.txt (captured go test -cover ./... run). Packages with 0.0% of statements and no ok/elapsed-time line have **no test files**.

Package	Coverage	Note
build	0.0%	no test files
cmd/api-server	0.0%	no test files
cmd/challenge-runner	0.0%	no test files
cmd/grpc-server	0.0%	no test files
cmd/monitor-server	0.0%	no test files
cmd/ssh-translation	0.0%	no test files
cmd/translator	0.0%	no test files
pkg/challenge_runner	0.0%	no test files
pkg/grpc	0.0%	no test files
pkg/grpc/proto	0.0%	no test files (generated)
pkg/hash	0.0%	no test files
tools + tools/working/* (13 pkgs)	0.0%	no test files (dev scripts)
test/security	0.0%	harness pkg (ok, 0.0%)
cmd/unified-translator	1.8%	PRIMARY CLI — critically low
cmd/preparation-translator	9.1%	low
test/utils	11.3%	harness helper
test/mocks	12.1%	harness mocks
cmd/translate-ssh	16.3%	low
cmd/markdown-translator	16.9%	low
pkg/sshworker	19.6%	low — distributed worker core
cmd/ebook-translator	21.8%	low
cmd/server	24.4%	low
pkg/storage	32.9%	low — PostgreSQL/Redis/SQLite abstraction
pkg/hardware	44.6%	moderate
internal/config	47.4%	moderate
cmd/cli	50.2%	moderate
pkg/api	55.0%	moderate
cmd/deployment	55.3%	moderate
pkg/verification	64.0%	moderate
pkg/format	72.2%	good
pkg/distributed	73.2%	good
pkg/markdown	73.9%	good
pkg/websocket	76.1%	good
pkg/batch	77.2%	good
pkg/ebook	77.6%	good
pkg/translator/llm	77.9%	good — LLM provider layer
pkg/translator	79.0%	good — core translation pipeline

Package	Coverage	Note
pkg/language	79.6%	good
pkg/fb2	80.1%	good
internal/verifier/persistence	82.5%	good
pkg/coordination	83.4%	good
pkg/version	83.6%	good
pkg/logger	84.3%	good
pkg/security	84.8%	TEST FAIL (see below)
internal/verifier	86.4%	good
pkg/models	88.6%	good
pkg/preparation	89.2%	good
internal/verifier/discovery	90.4%	excellent
internal/verifier/selection	92.3%	excellent
pkg/report	97.7%	excellent
internal/cache	98.1%	excellent
pkg/events	100.0%	full
pkg/progress	100.0%	full
pkg/script	100.0%	full
internal/services	100.0%	full
internal/verifier/scoring	100.0%	full

Harness/aggregate packages under test/ (no coverage attributable)

Package	Result
test	0.0% (no statements)
test/distributed	ok, [no statements]
test/integration	ok, [no statements]
test/translator, test/translator/llm	ok, [no statements]
test/unit	ok, [no statements]
tests	ok, [no statements]

(These run real cross-package assertions but attribute their coverage to the packages-under-test, so they self-report [no statements].)

Failing package (finding, captured)

```

--- FAIL: TestAuthService_TokenExpirationEdgeCases (0.05s)
    --- FAIL: TestAuthService_TokenExpirationEdgeCases/Short_TTL
    (0.05s)
        auth_test.go:713: Received unexpected error:

```

```
token has invalid claims: token is
expired
    auth_test.go:714: Expected value not to be nil.
FAIL
coverage: 84.8% of statements
FAIL    digital.vasic.translator/pkg/security    11.375s
```

This is a **TEST FAIL** (not a build fail): pkg/security compiles and most of its suite passes (84.8% coverage achieved), but TestAuthService_TokenExpirationEdgeCases/Short_TTL is a flaky/timing-sensitive JWT-expiry test — the short-TTL token expires before the assertion runs, so the verify returns token is expired. This makes the whole-module go test ./... exit non-zero. No package in the main module fails to BUILD.

Test-type matrix (constitution-mandated set)

Detection via: //go:build tags, filenames, the test/ subtree, benchmark functions, and tests/banks/. Scoped to the main module.

Test type	Present?	Where	Approx. count
unit	YES	*_test.go alongside every pkg/*, internal/*; test/unit/	15 files in test/unit/ + majority of ~80 package _test.go files
integration	YES	//go:build integration: test/integration/ (5 files), test/distributed/integration_test.go; pkg/translator/integration_test.go	~6 tagged files
e2e	YES	//go:build e2e: test/e2e/verified_translation_e2e_test.go, test/e2e/translation_quality_e2e_test.go	2 files
performance	YES	//go:build performance: test/performance/ (2); pkg/distributed/performance*_test.go, pkg/translator/performance_test.go	~5 files
stress	YES	//go:build stress: test/stress/ (2), test/distributed/stress_test.go	3 files
security	YES	//go:build security: test/security/ (4), test/distributed/security_test.go; pkg/security/*_test.go, pkg/translator/	~10 files

Test type	Present?	Where	Approx. count
benchmark	YES	security_test.go, pkg/distributed/security_test.go func Benchmark* across ~58 main-module _test.go files (events, language, hardware, translator, storage, security, websocket, verification, ...)	~58 files contain Benchmarks
distributed	YES	test/distributed/ (6 files, tagged integration/security/stress/performance)	6 files
Challenges	PARTIAL	tests/banks/*.yaml (full-qa-{api,web,cli}.yaml, fixes-validation.yaml) + cmd/challenge-runner/ + pkg/challenge_runner/ (0% coverage, no Go tests)	4 banks + runner
chaos	NO	—	0
ddos	NO	(rate-limit tests in pkg/security/ratelimit_test.go exist but no DDoS load test)	0
scaling	NO	(no *scale*/scaling load test in main module)	0
full-automation	NO	(no *automation*/*autonomous* Go test in main module; HelixQA banks are external)	0
ui	NO	—	0
ux	NO	—	0
HelixQA	EXTERNAL	helix_qa/ is a separate submodule/module; banks in tests/banks/ reference it but no in-module Go driver	n/a in this module

Build tags actually present (captured): integration, e2e, performance, stress, security, plus demo and ignore on non-test files.

Top gaps (prioritized)

A. Core packages with 0% / critically-low coverage

1. **cmd/unified-translator** — **1.8%**. This is the PRIMARY user-facing CLI per the project README. Near-zero coverage on the main entry point is the single highest-risk gap.
2. **pkg/grpc + pkg/grpc/proto + cmd/grpc-server** — **0%**. The entire gRPC service surface (proto: pkg/grpc/translator.proto) is untested.
3. **cmd/api-server + cmd/monitor-server + cmd/server (24.4%)**. REST/WebSocket and monitoring servers — cmd/api-server/cmd/monitor-server have no tests at all.
4. **pkg/sshworker** — **19.6%** and **cmd/ssh-translation / cmd/translate-ssh (0% / 16.3%)**. The SSH distributed-worker path is thinly covered.

5. **pkg/storage** — **32.9%**. PostgreSQL/Redis/SQLite + translation cache abstraction — low for a data-integrity-critical layer.
6. **pkg/hash** — **0%**, **pkg/challenge_runner** — **0%**. Untested supporting packages.

(Note: the *core translation* packages are actually in good shape — pkg/translator 79.0%, pkg/translator/llm 77.9%, pkg/ebook 77.6%, pkg/fb2 80.1%, pkg/format 72.2%, internal/verifier/* 82–100%. The gaps concentrate in **entry-point binaries (cmd/*)**, **RPC/server surfaces**, and **SSH/storage infra**.)

B. Mandated test types ENTIRELY MISSING (main module)

- **chaos** — none (constitution §11.4.85 mandates chaos suites per fix).
- **ddos** — none.
- **scaling** — none.
- **full-automation** — no in-module Go full-automation driver (HelixQA is referenced only via external tests/banks/*.yaml).
- **ui / ux** — none (the project ships HTML dashboards monitor.html etc. but they have no UI/UX test).

C. Reliability finding

- pkg/security/TestAuthService_TokenExpirationEdgeCases/Short_TTL FAILs on a short-TTL JWT timing race (token is expired). The whole-module go test ./... exits non-zero because of it. This is a real, reproducible failure captured this session, not flaked-away.

Every coverage number and the FAIL above is taken verbatim from /tmp/_cov_run.txt, /tmp/cov.out, and the go tool cover -func total line captured in this session.